

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks:50

Annexure A

Coding challenge

Code of Conduct:

*I **C.BHUVANESH (192521108)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

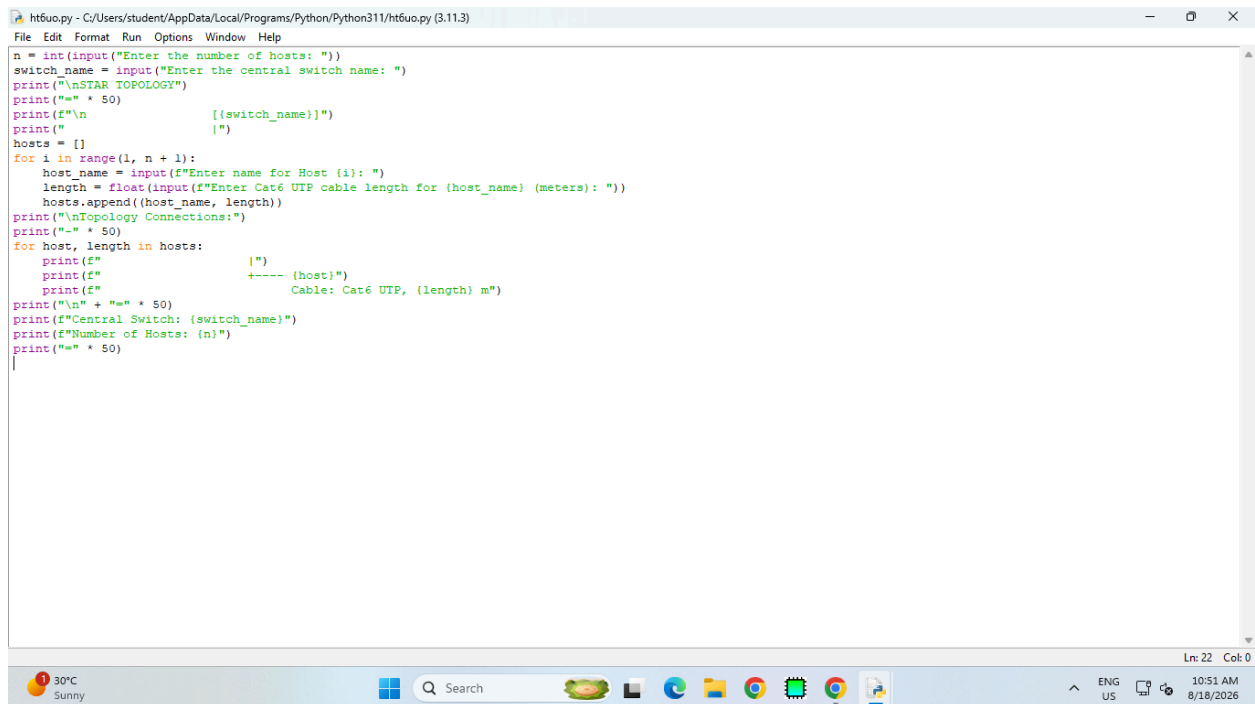
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: c.bhuvanesh

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.



```
ht6uo.py - C:/Users/student/AppData/Local/Programs/Python/Python311/ht6uo.py (3.11.3)
File Edit Format Run Options Window Help

n = int(input("Enter the number of hosts: "))
switch_name = input("Enter the central switch name: ")
print("\nSTAR TOPOLOGY")
print("=" * 50)
print(f"\n          [{switch_name}]\n")
print("          |")
hosts = []
for i in range(1, n + 1):
    host_name = input(f"Enter name for Host (i): ")
    length = float(input(f"Enter Cat6 UTP cable length for {host_name} (meters): "))
    hosts.append((host_name, length))
print("\nTopology Connections:")
print("=" * 50)
for host, length in hosts:
    print(f"          |")
    print(f"          +---- {host}")
    print(f"          Cable: Cat6 UTP, (length) m")
print("\n" + "=" * 50)
print(f"Central Switch: {switch_name}")
print(f"Number of Hosts: {n}")
print("=" * 50)
|
```

Ln: 22 Col: 0

30°C Sunny

Search

ENG US 10:51 AM 8/18/2026

```
IDLE Shell 3.11.3
File Edit Shell Debug Options Window Help
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr 4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:/Users/student/AppData/Local/Programs/Python/Python311/ht6uo.py ==
Enter the number of hosts: 4
Enter the central switch name: sw

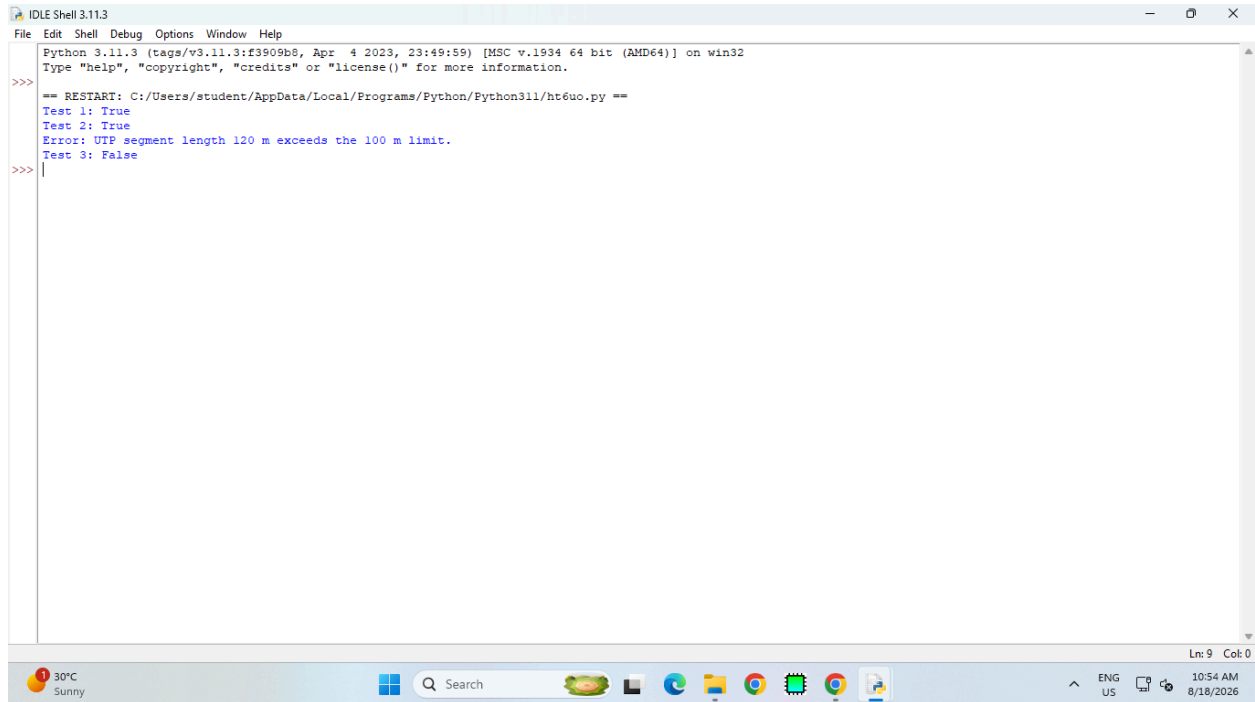
STAR TOPOLOGY
=====
[sw]
|
Enter name for Host 1: 1
Enter Cat6 UTP cable length for 1 (meters): 1
Enter name for Host 2: 2
Enter Cat6 UTP cable length for 2 (meters): 12
Enter name for Host 3: 3
Enter Cat6 UTP cable length for 3 (meters): 14
Enter name for Host 4: 4
Enter Cat6 UTP cable length for 4 (meters): 16

Topology Connections:
=====
|
+---- 1
|      Cable: Cat6 UTP, 1.0 m
|
+---- 2
|      Cable: Cat6 UTP, 12.0 m
|
+---- 3
|      Cable: Cat6 UTP, 14.0 m
|
+---- 4
|      Cable: Cat6 UTP, 16.0 m
|
=====
Central Switch: sw
Number of Hosts: 4
=====
>>>
```

2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.

```
ht6uo.py - C:/Users/student/AppData/Local/Programs/Python/Python311/ht6uo.py (3.11.3)
File Edit Format Run Options Window Help
def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        print(f"Error: UTP segment length (length_m) m exceeds the 100 m limit.")
        return False

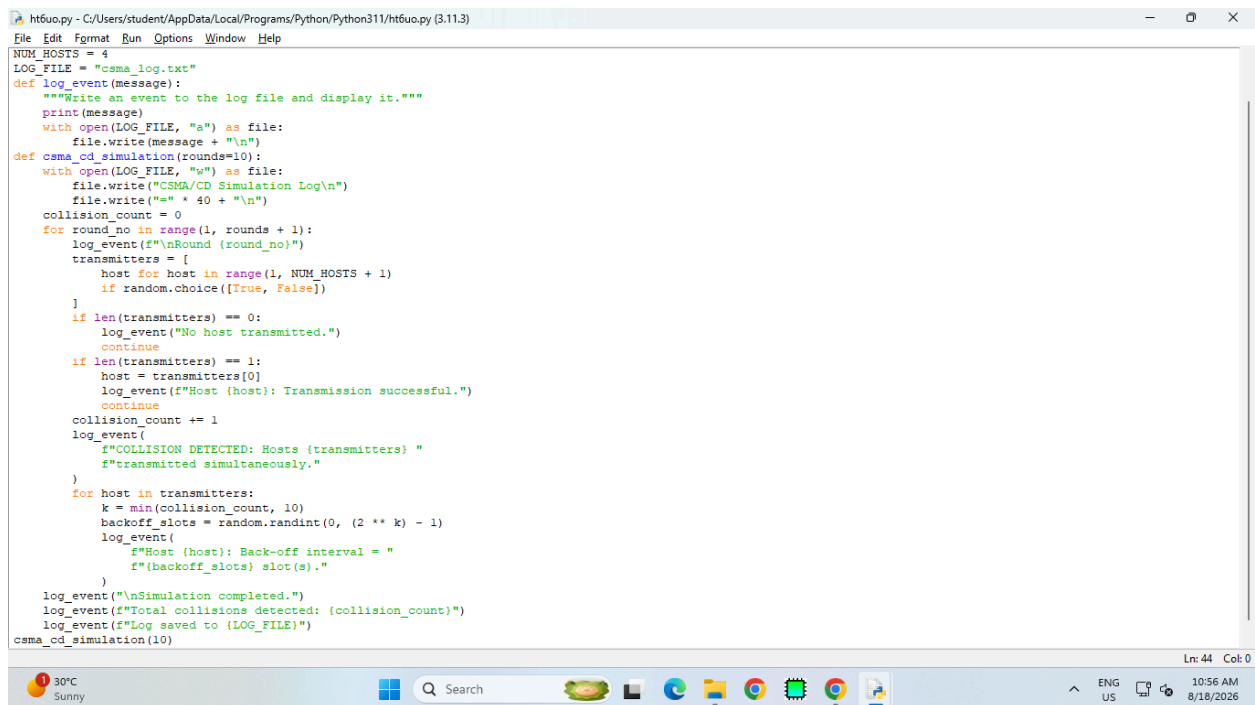
print("Test 1:", validate_segment(50))
print("Test 2:", validate_segment(100))
print("Test 3:", validate_segment(120))
```



```
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr 4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
== RESTART: C:/Users/student/AppData/Local/Programs/Python/Python311/ht6uo.py ==
Test 1: True
Test 2: True
Error: UTF segment length 120 m exceeds the 100 m limit.
Test 3: False
>>>
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma\_log.txt`.



```
ht6uo.py - C:/Users/student/AppData/Local/Programs/Python/Python311/ht6uo.py (3.11.3)
File Edit Format Run Options Window Help
NUM_HOSTS = 4
LOG_FILE = "csma_log.txt"
def log_event(message):
    """Write an event to the log file and display it."""
    print(message)
    with open(LOG_FILE, "a") as file:
        file.write(message + "\n")
def csma_cd_simulation(rounds=10):
    with open(LOG_FILE, "w") as file:
        file.write("CSMA/CD Simulation Log\n")
        file.write("=" * 40 + "\n")
    collision_count = 0
    for round_no in range(1, rounds + 1):
        log_event(f"Round {round_no}")
        transmitters = [
            host for host in range(1, NUM_HOSTS + 1)
            if random.choice([True, False])
        ]
        if len(transmitters) == 0:
            log_event("No host transmitted.")
            continue
        if len(transmitters) == 1:
            host = transmitters[0]
            log_event(f"Host {host}: Transmission successful.")
            continue
        collision_count += 1
        log_event(
            f"COLLISION DETECTED: Hosts {transmitters} "
            f"transmitted simultaneously."
        )
        for host in transmitters:
            k = min(collision_count, 10)
            backoff_slots = random.randint(0, (2 ** k) - 1)
            log_event(
                f"Host {host}: Back-off interval = "
                f"{backoff_slots} slot(s)."
            )
        log_event("\nSimulation completed.")
        log_event(f"Total collisions detected: {collision_count}")
        log_event(f"Log saved to {LOG_FILE}")
csma_cd_simulation(10)
```

```
IDLE Shell 3.11.3
File Edit Shell Debug Options Window Help
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr 4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:/Users/student/AppData/Local/Programs/Python/Python311/ht6uo.py ==

Round 1
No host transmitted.

Round 2
No host transmitted.

Round 3
Host 4: Transmission successful.

Round 4
COLLISION DETECTED: Hosts [1, 3, 4] transmitted simultaneously.
Host 1: Back-off interval = 1 slot(s).
Host 3: Back-off interval = 1 slot(s).
Host 4: Back-off interval = 0 slot(s).

Round 5
COLLISION DETECTED: Hosts [2, 4] transmitted simultaneously.
Host 2: Back-off interval = 3 slot(s).
Host 4: Back-off interval = 0 slot(s).

Round 6
COLLISION DETECTED: Hosts [1, 3, 4] transmitted simultaneously.
Host 1: Back-off interval = 3 slot(s).
Host 3: Back-off interval = 1 slot(s).
Host 4: Back-off interval = 7 slot(s).

Round 7
COLLISION DETECTED: Hosts [2, 3] transmitted simultaneously.
Host 2: Back-off interval = 10 slot(s).
Host 3: Back-off interval = 0 slot(s).

Round 8
COLLISION DETECTED: Hosts [1, 2, 3, 4] transmitted simultaneously.
Host 1: Back-off interval = 11 slot(s).
Host 2: Back-off interval = 22 slot(s).
Host 3: Back-off interval = 3 slot(s).
```

4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

```
*switch_server.py - C:/Users/student/AppData/Local/Programs/Python/Python311/switch_server.py (3.11.3)
File Edit Format Run Options Window Help
import socket
HOST = "127.0.0.1"
PORT = 5000
SWITCH_MAC = "AA:AA:AA:AA:AA:AA"
HOST1_MAC = "00:11:22:33:44:01"
HOST2_MAC = "00:11:22:33:44:02"
mac_table = {}

def print_mac_table():
    print("\n----- SWITCH MAC ADDRESS TABLE -----")
    print("MAC Address      Port")
    print("-----")
    for mac, port in mac_table.items():
        print(f"{mac}<20] {port}")
    print("-----")

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Star Switch started...")
print("Switch MAC Address:", SWITCH_MAC)
print("Waiting for Host 1 connection...")
connection, address = server.accept()
print("Host 1 connected.")
while True:
    data = connection.recv(1024)
    if not data:
        break
    message = data.decode()
    mac_table[HOST1_MAC] = "Port 1"
    print("\nFrame received by switch")
    print("Source MAC:", HOST1_MAC)
    print("Destination MAC:", HOST2_MAC)
    print("Message:", message)
    print("Switch forwarding frame from Port 1 to Port 2")
    mac_table[HOST2_MAC] = "Port 2"
    print_mac_table()
    response = "Message received by Host 2"
    connection.send(response.encode())
connection.close()
server.close()
```

```
*host1_client.py - C:/Users/student/AppData/Local/Programs/Python/Python311/host1_client.py (3.11.3)*
File Edit Format Run Options Window Help

import socket
HOST = "127.0.0.1"
PORT = 5000
HOST1_MAC = "00:11:22:33:44:01"
HOST2_MAC = "00:11:22:33:44:02"
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("Host 1 connected to the star switch.")
print("Host 1 MAC:", HOST1_MAC)
print("Host 2 MAC:", HOST2_MAC)
while True:
    message = input("\nEnter message (type 'exit' to stop): ")
    if message.lower() == "exit":
        break
    print("\nHost 1 sending frame...")
    print("Source MAC:", HOST1_MAC)
    print("Destination MAC:", HOST2_MAC)
    client.send(message.encode())
    response = client.recv(1024).decode()
    print("Host 1 received:", response)
client.close()
```

Ln: 21 Col: 0

30°C Sunny

Search

ENG US 11:04 AM 8/18/2026

5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

```
*dg.py - C:/Users/student/AppData/Local/Programs/Python/Python311/dg.py (3.11.3)*
File Edit Format Run Options Window Help

class StarSwitch:
    def __init__(self):
        self.ports = []
    def add_port(self, host_id):
        if host_id not in self.ports:
            self.ports.append(host_id)
            print(f"Host {host_id} connected to the switch.")
        else:
            print(f"Host {host_id} is already connected.")
    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(f"Host {host_id} disconnected from the switch.")
        else:
            print(f"Host {host_id} is not connected.")
    def broadcast(self, frame):
        source = frame["source"]
        message = frame["message"]
        print(f"\nBroadcasting frame from Host {source}: {message}")
        for port in self.ports:
            if port != source:
                print(f"Frame delivered from Host {source} to Host {port}")

switch = StarSwitch()
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")
frame = {
    "source": "Host1",
    "message": "Hello, all hosts!"
}
switch.broadcast(frame)
switch.remove_port("Host4")
frame2 = {
    "source": "Host2",
    "message": "Hello from Host2!"
}
switch.broadcast(frame2)
```

Ln: 38 Col: 0

30°C Sunny

Search

ENG US 11:06 AM 8/18/2026

The screenshot shows a Windows 10 desktop. In the foreground, a Python IDLE Shell window is open, displaying the following text:

```

Python 3.11.3 (tags/v3.11.3:f3909b8, Apr  4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
==== RESTART: C:/Users/student/AppData/Local/Programs/Python/Python311/dg.py ====
Host Host1 connected to the switch.
Host Host2 connected to the switch.
Host Host3 connected to the switch.
Host Host4 connected to the switch.

Broadcasting frame from Host Host1: Hello, all hosts!
Frame delivered from Host Host1 to Host Host2
Frame delivered from Host Host1 to Host Host3
Frame delivered from Host Host1 to Host Host4
Host Host4 disconnected from the switch.

Broadcasting frame from Host Host2: Hello from Host2!
Frame delivered from Host Host2 to Host Host1
Frame delivered from Host Host2 to Host Host3
>>>

```

In the bottom right corner, a Snipping Tool notification box is visible, stating: "Screenshot copied to clipboard. Automatically saved to screenshots folder. Markup and share". The Windows taskbar at the bottom shows the date and time as 11:06 AM on 8/18/2026, along with system icons for temperature (30°C Sunny) and language (ENG US).

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done